

Developer Guide

This guide provides the necessary information for the maintenance of the GenESyS OVA.

The OVA was created to provide a standardized, reproducible, and lightweight environment for running and developing GenESyS in UFSC laboratories. It encapsulates the operating system, dependencies, automation scripts, and continuous integration with the project repository.

1 General Characteristics of the OVA

The OVA structure is composed of:

- Debian netinst operating system with LXDE + LightDM graphical environment
- Automated setup scripts
- Incremental update scripts
- Automatic GitHub integration
- QtCreator development environment
- Web service running via systemd
- Automatic update of GenESyS releases

2 Environment Provided by the OVA for GenESyS Development

2.1 Operating System

Base distribution: Debian netinst

Graphical interface: LXDE

Display manager: LightDM

2.2 Minimum Virtual Machine Resources Required for OVA Execution

Resource	Value
CPU	4 cores
RAM	4 GB
Storage	8 GB

2.3 Installed Development Tools

The environment automatically installs:

- Compilation and build
- GCC / G++
- build-essential
- CMake
- Ninja
- Frameworks
- Qt6
- IDE-QtCreator
- Auxiliary tools
- Graphviz
- R
- NGSpice

2.4 The setup configures:

- LXDE
- Xorg
- LightDM
- Firefox ESR
- ABNT2 keyboard
- keyboard shortcuts
- XTerm terminal

2.5 OVA Scripts Structure

The OVA functions using 4 scripts:

- **min_setup_qtcreator.sh**: performs the initial configuration of the operating system, installing dependencies, graphical interface, necessary tools, and adjusting basic settings (keyboard, browser, shortcuts, and system cleanup).
- **init.sh**: executed at the beginning of the user session and manages the installation and update of GenESyS, including downloading new versions, configuring the execution environment, and controlling the development repository.
- **update.sh**: is responsible for updating the OVA itself, applying version migrations, correcting old configurations, and ensuring that the environment is consistent with the latest defined version.

- **install_guest_add.sh:** automates the installation of VirtualBox Guest Additions, allowing better integration between the virtual machine and the host, such as video, mouse support, and resource sharing.

2.5.1 Minimum Setup Scripts (min_setup_qtcreator.sh)

The minimum setup scripts constitute the main automation mechanism for the environment configuration present in the OVA, being responsible for transforming a basic Debian system installation into a completely functional environment for educational use with the GenESyS Simulator. This script was developed with a focus on reproducibility, standardization, and reduction of manual intervention during the machine preparation process.

The script structure follows a modular organization based on functions, each responsible for a specific stage of system configuration. Initially, an administrative privilege verification is performed, ensuring the script is executed with adequate permissions for package installation and modification of system settings. Then, configurable variables are defined, such as default user, directories, and optimization parameters, allowing greater flexibility in execution.

Among the first steps executed is user configuration and installation of the sudo utility, ensuring the virtual machine's default user can execute administrative commands. Next, the script installs the graphical environment using the Xorg server in conjunction with the LXDE environment and the LightDM login manager.

Another central point of the script is the installation of dependencies required for the development and execution of the GenESyS Simulator. This includes compilers, build tools such as CMake and Ninja, Qt6 libraries, as well as auxiliary tools such as Graphviz, R, and NGSpice.

In addition to the basic configuration, the script also implements usability-focused adjustments, such as defining the default browser (Firefox ESR), configuring the Brazilian keyboard layout (ABNT2), and creating keyboard shortcuts to facilitate system interaction. A mechanism responsible for executing a remote script that performs the installation and update of GenESyS Simulator and the OVA when the user session starts is also configured.

At the end of execution, the script performs system cleanup procedures, removing unnecessary packages and residual files. Additionally, the zero-fill

technique is applied, filling free space with zeros to improve the OVA compression rate and reduce its final size.

2.5.2 GenESyS Initialization Script (init.sh)

The initialization script aims to automate the process of installation, update, and integration of the GenESyS Simulator into the user environment. This script is executed automatically at the start of the user session.

Initially, the script uses the *-euo pipefail* options combined with *set -x* to stop execution immediately in case of command errors. All standard output and errors are redirected and saved in a log file stored in the system's temporary directory (*/tmp/genesys_install.log*), using the *tee* command to allow tracking of eventual failures.

Before any network operation, the script performs an active internet connectivity check, executing a loop that waits until the system can resolve the GitHub address via *getent hosts*. This step is fundamental, as the entire subsequent process depends on access to the remote repository and the project's API.

Next, the *script* manages the development environment configuration. If the *branch* configuration file (*.genesys_dev_branch*) does not exist, the *gxmessage* visual interface is invoked so that the user explicitly chooses between the *currentStable* or *WorkInProgress* branch. If the local development repository is not yet present in the configured path, the script displays a temporary visual warning and clones the selected branch, saving this initial state in the *genesys.lastAppliedBranch* settings.

Parallel to the development repository, the *script* manages the end-user version by consuming the *GitHub Releases API* directly. It extracts the tag of the latest version available and compares it with the locally registered version. In case of divergence or first installation, the *script* displays a visual notification, downloads the compressed package (*tar.gz*), preventively stops the running *web* service, and extracts the new binaries.

During this end-user update stage, the graphical application and *web* server binaries are moved to the user's local executable directory (*~/.local/bin/*), receiving execution permissions. The system icon is copied to the local icons directory and a *.desktop* initialization shortcut is generated in *~/.local/share/applications/*, integrating *GenESySQt* into the graphical environment application menu. Additionally, the script

creates and configures a *systemd* unit file at the user level (*genesys-web.service*), reloads the *daemon*, and initializes the *web* server to run in the background with an automatic restart policy.

Finally, the script performs strict integrity validations on the development repository before any automated update. It detects if the repository is in a *detached HEAD* state or if there are unsaved local changes, blocking the automatic update in these scenarios to avoid loss of the developer's work. The script also evaluates if there was a manual change in the *branch* configuration or if the user performed a divergent local *checkout*. If the environment is clean and consistent, a commit comparison (*merge-base*) is made between the local and remote state. If there is a linear advance available in the remote branch, the user is consulted via the graphical interface to authorize the *git pull*; if a divergence conflict is detected, the automatic update is aborted and a manual resolution instruction is displayed.

2.5.3 OVA Update Script (*update.sh*)

In addition to GenESyS Simulator management, the environment includes a dedicated mechanism for updating the OVA itself. This mechanism is implemented through a versioning script, which allows applying modifications to the system incrementally and in a controlled manner.

The operation is based on maintaining a file that stores the currently installed version of the environment. Whenever the script is executed, this version is compared with the target version defined in the code. If the target version is higher, the system identifies which updates need to be applied.

Each update is implemented as a specific function, associated with a system version. This modular approach allows adding new updates in an organized way, without impacting previous versions. Furthermore, using the *dpkg* tool for version comparison ensures consistency in the application of updates.

During execution, the script goes through a list of versions and applies only those that have not yet been executed in the system. After each update is applied, the corresponding version is recorded, avoiding unnecessary re-executions.

This incremental update model eliminates the need for complete redistribution of the OVA whenever modifications are made. Instead, small updates can be applied directly to the already installed environment, making environment maintenance easier.

Additionally, this script is integrated into a system service managed by systemd, which ensures its automatic execution during startup. The service is configured with retry policies in case of failure.

2.5.4 VirtualBox Guest Additions Installation Script

As part of the virtual environment preparation, an auxiliary script responsible for installing VirtualBox Guest Additions was included in the repository. This component is fundamental to improve integration between the host system and the virtual machine, offering resources such as better graphical support, automatic screen resizing, video acceleration, and clipboard sharing.

The script was designed to be executed manually with administrative privileges, receiving the VirtualBox version used as a parameter. This approach ensures compatibility between the Guest Additions version and the virtualization environment, avoiding problems arising from version incompatibilities.

Initially, the script performs a check to ensure it is being executed as the root user. Then, it validates the presence of the argument corresponding to the VirtualBox version, interrupting execution if this information is not provided. This validation is essential for the correct construction of the download URL for the ISO image containing the Guest Additions.

In the next stage, the script automatically installs the necessary packages for compiling the kernel modules, including tools such as build-essential, dkms, and the kernel headers in use. After environment preparation, the script downloads the ISO image corresponding to the specified VirtualBox version directly from the official repository. Then, this image is mounted in a temporary directory, allowing access to the installer contained in the media.

The installation process is then executed via the VBoxLinuxAdditions.run script, responsible for compiling and installing the necessary modules on the guest system. Upon completion, the script unmounts the ISO image and removes temporary files, keeping the system clean. Finally, a message is displayed advising to restart the virtual machine, a necessary step for the changes to take full effect.

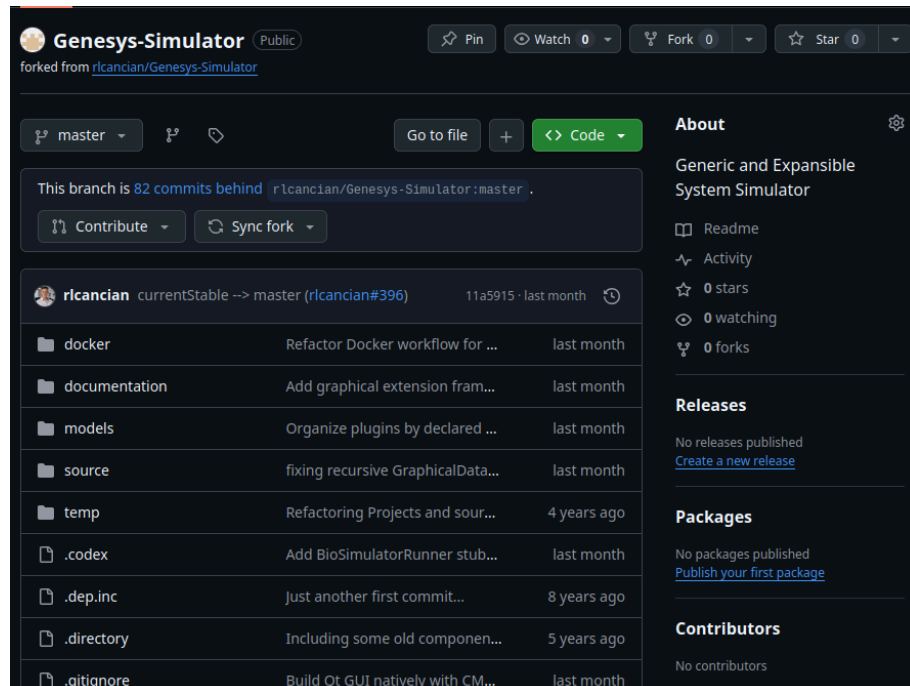
2.6 How to Update the OVA

The objective of this section is to briefly describe how to perform updates, for users, developers, and updates related to new versions of the OVA environment.

2.6.1 Updating the Release for Users

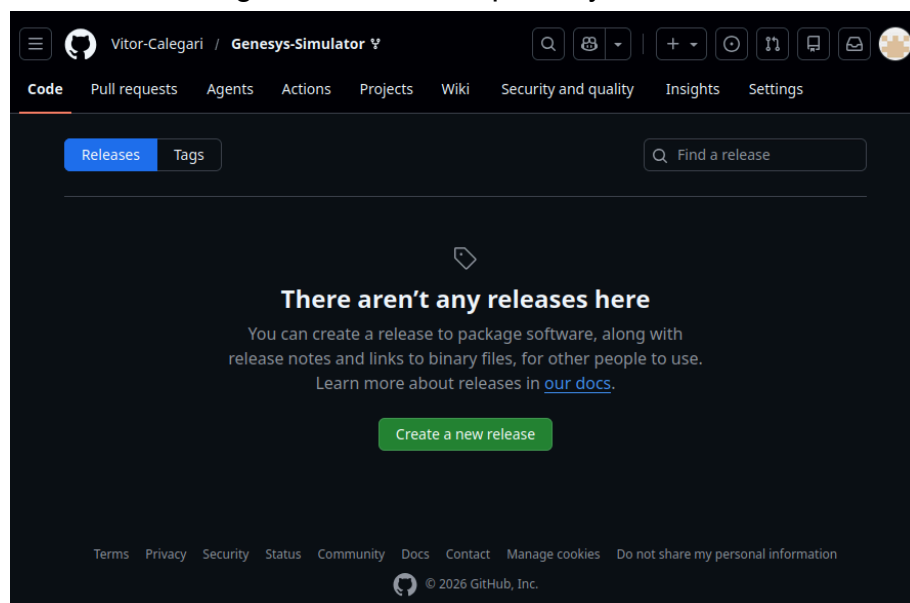
To perform a new release, from a repository without releases, one must select the “Releases” tab on the repository home page, and later “Create a new release”.

GitHub home page



Source: Created by the authors

Page dedicated to repository releases



Source: Created by the authors

After that, create a tag, define a title and a description. Gather in an empty directory the binary files of the QT interface (`genesys_qt_gui_application`) and the web server (`genesys_web_app`), along with the program icon (`genesysico.gif`), and compress the previously mentioned files by executing in the directory that gathers the three cited files: `tar -czf genesys-linux.tar.gz genesys_qt_gui_application genesys_web_app genesysico.gif`.

Page “New Release”

Source: Created by the authors

Finish by uploading the generated *tar.gz* to GitHub and click “Publish release”. Note that the “*Release Label*” on the first *release* creation should be “None”, on subsequent creations, it should be selected as “Latest”.

Note: It is essential that the executable files be strictly named as “`genesys_qt_gui_application`” and “`genesys_web_app`”. Furthermore, to ensure compatibility, it is recommended that these binaries be generated in a GenESyS OVA.

2.6.2 Updating the OVA Version

To perform an update in the OVA version, simply access the “`update.sh`” *script* available in the `ova` directory, increment the value of the “`CURRENT_VERSION`”

variable, and add an “update_x_y” type function with the desired modifications to the OVA environment. Furthermore, it is also necessary to add the newly created version to the version loop in the *run_updates* function. Every update is made incrementally (Ex: If updates 1.0, 1.1, 1.2, 1.3 are available, and the current OVA is in version 1.1, functions 1.2 and 1.3 will be executed sequentially). Note that codes executed during updates are run as “root”. To exemplify, an “update” function for version 1.0 is available in the repository by default.

2.7 Developer Observations

2.7.1 Web server:

The GenESyS web server works in the background as a systemd user service and starts automatically as soon as you log in to the system, without needing superuser privileges. The server executable is stored in the hidden directory *~/.local/bin/*. The executable is obtained from the latest release available in the repository. So the system knows how to control this server, the script creates a service configuration file in *~/.config/systemd/user/genesys-web.service*.

To stop the server, run:

```
systemctl --user stop genesys-web.service
```

To start the server, run:

```
systemctl --user start genesys-web.service
```

To disable automatic server execution when starting the VM, run:

```
systemctl --user disable genesys-web.service
```

To enable automatic server execution when starting the VM, run:

```
systemctl --user enable genesys-web.service
```

2.7.2 File Locations:

In the common user scope (vboxuser), the initialization script (init.sh) is downloaded in *~/.local/bin/genesys_startup.sh* and configured to execute after the user performs their *login* in the graphical interface. This occurs via a *.desktop*

created in **~/.config/autostart/**. At the system level (as root), the utility for guest additions is kept in **/usr/local/bin/install_guest_add**, while the virtual machine updater (ova_update_runner) is in **/usr/local/bin/** and is executed automatically during system startup by the systemd manager, which tries to execute it as soon as the network is active and insists in case of failure. The OVA version is stored in **/etc/genesys_ova_version**, the branch used in the repository located at **~/Documents/Genesys-Dev** is defined in **~/.genesys_dev_branch**, and the installed GenESyS release version, used for control, is registered in **~/.genesys_user_version**.